

THYMESIA

프로젝트 이름

티메시아 팀프로젝트

제작 기간

2025.02 ~ 2025.04

개발 인원

6명

개발 환경

Visual Studio 2022, DirectX11, HLSL, ImGui

담당 파트

맵 툴, 클라이언트 콘텐츠 개발, 최적화

동영상 링크



[시연 영상 보기](#)

깃헙 링크



[깃헙 바로가기](#)



PLAN

개발 일정

	맵 툴 / 네비게이션	메쉬 인스턴싱	프러스텀 컬링	아이템 시스템 설계	상호작용 시스템 설계	GS셰이더 연출	이펙트 / 사운드
1주차 (2/25~3/02)	맵 툴 프로젝트 제작						
2주차 (3/03~3/09)	네비게이션 시스템 완성						
3주차 (3/10~3/16)	편집 작업 및 바이너리화	인스턴싱 구조 설계					
4주차 (3/17~3/20)	데이터파일 배포	렌더링 최적화 진행	컬링, 인스턴싱 최적화				
5주차 (3/22~3/28)				아이템 매니저 설계			
6주차 (3/29~4/06)				아이템 상호작용	상호작용 시각 연출 구현		
7주차 (4/07~4/15)						GS기반 시각 연출 구현	
8주차 (4/17~4/23)							이펙트, 사운드 추가 작업

[1] Navigation System

1

Navigation Mesh

1

네비게이션 툴의 필요성

2

설계 구조 및 사용

3

결과 및 최적화 기여 부분

2

Edge Navigation

1

엣지 네비게이션의 필요성

2

구현 과정

3

슬라이딩 벡터

3

픽셀 피킹

1

픽셀 피킹의 필요성

2

구현 과정

3

결과 및 최적화 기여 부분

[2] Geometry Shader 연출

4

오브젝트 연출

1

극적인 연출의 필요성

2

수학, 물리 공식 적용

3

모델의 정점 정보 수정

4

동일 메쉬 복제

5

결과 및 기여 부분

[3] 오브젝트와 상호작용

5

상호작용

1

상호작용 시각화의 필요성

2

상호작용 UI Text

3

Rim Light 셰이더

4

몬스터와 빔-라인 연결

[4] 기타 최적화 작업

6

메쉬 인스턴싱

1

인스턴싱의 필요성과 흐름

2

인스턴싱 구조 설계

3

결과 및 최적화 기여 부분

7

AABB 절두체 컬링

1

컬링의 필요성

2

AABB - 절두체 컬링

3

결과 및 최적화 기여 부분

4

기타 최적화 (맵 구역 분리)

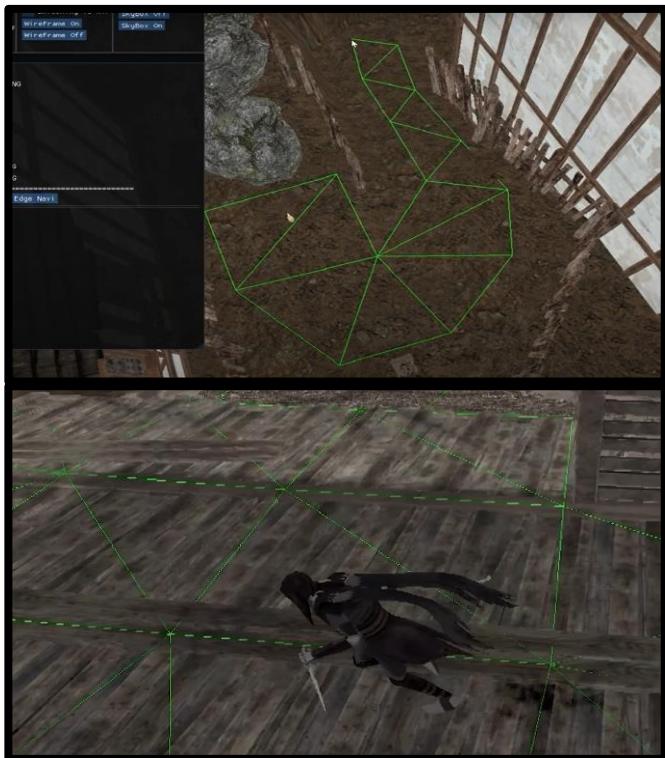


1-1

Navigation Tool의 필요성

◆ 넓은 월드 맵을 움직일 때, 플레이어가 움직일 수 있는 영역을 직접 설정해주어 불필요한 충돌 연산을 피하고 싶었습니다.

◆ 따라서 게임 월드 스테이지 마다, 영역을 정해주고 최종적으로 bin 파일로 관리하는 네비게이션 툴 기능을 구현했습니다.



◆ 툴에서 Navigation 영역 편집 기능들입니다.

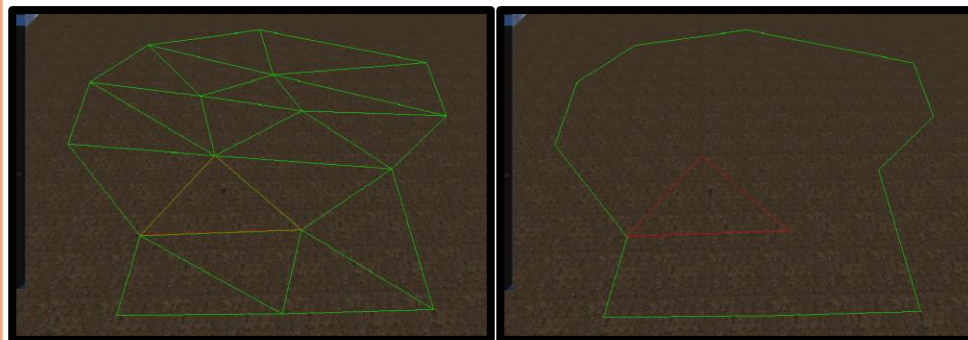
(1) 픽셀 피킹 기반으로 Cell 생성

(2) 이전 Cell 과 자동으로 이어지는 다음 Cell 생성

- 한 붓 그리기 형식으로 빠르게 길을 만들어낼 수 있습니다.
- 1번 Cell 과 2번 Cell의 꼭지점을 이어 Cell을 만들어낼 수 있습니다.

(3) Cell 삭제

(4) Edge Navigation 전환 - 슬라이딩 벡터 구현



(5) 플레이어와 가장 근접한 Cell의 인덱스 번호 반환

- 플레이어의 충간 이동 가능

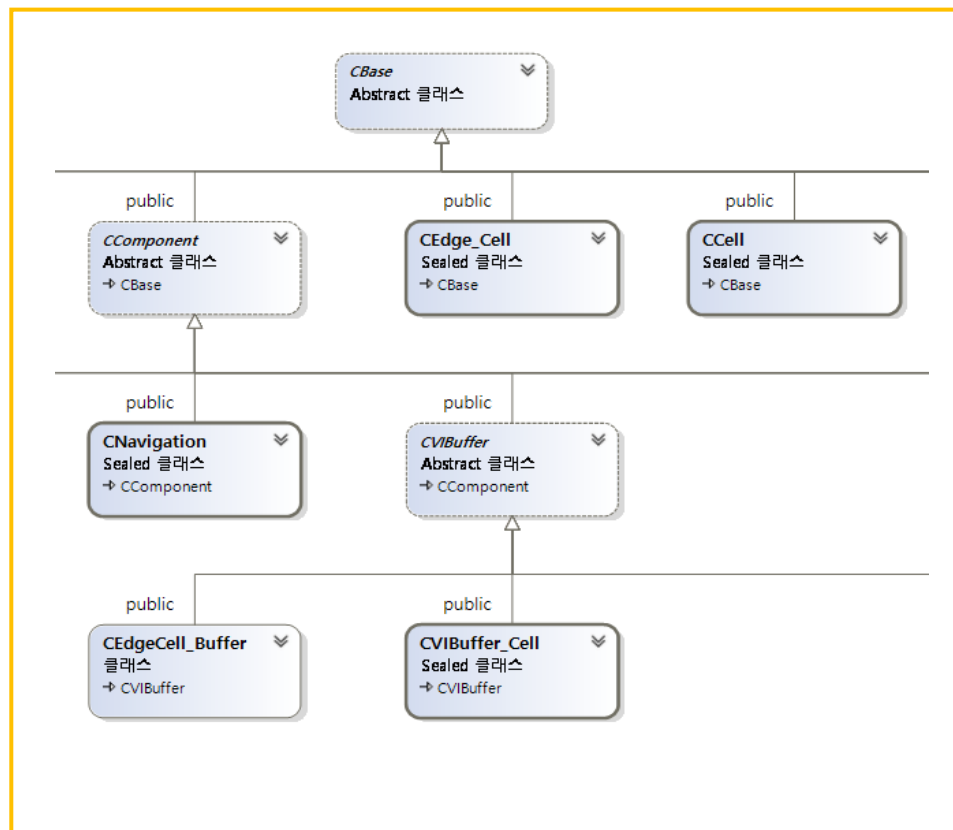
(6) 완성된 Navigation 영역 저장, 불러오기



1-2

설계 구조 및 사용

◆ CNavigation은 CComponent를 상속받은 Navigation 전용 컴포넌트로, GameObject에 부착되어 유닛들의 이동, 높이 보정, 슬라이딩 이동 처리 등을 담당합니다.



◆ 실제 게임 오브젝트 사용 예시

- Navigation 컴포넌트는 GameObject의 Transform 갱신 과정에서 Compute_Height() & isMove()를 통해 지형 대응 이동을 처리합니다.

```

// Player.cpp : Initialize()
Add_Component(LEVEL_GAMEPLAY,
TEXT("Prototype_Component_Navigation"),
TEXT("Com_Navigation"),
reinterpret_cast<CComponent*>(&m_pNavigationCom),
&Desc))
  
```

```

// Player.cpp : Update()
m_pTransformCom->Set_State(
CTransform::STATE_POSITION,
XMVectorSetY(vPosition,
m_pNavigationCom->Compute_Height(vPosition))
  
```

```

// Transfom.cpp : Go_Straight()
_vector vPosition = Get_State(STATE_POSITION);
_vector vLook = Get_State(STATE_LOOK);

vPosition += XMVector3Normalize(vLook) *
m_fSpeedPerSec * fTimeDelta;

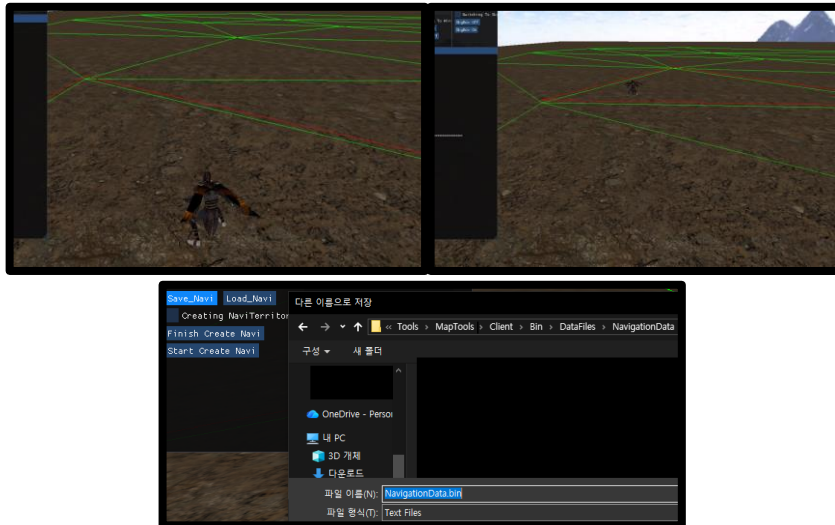
if (pNavigation->isMove(vPosition))
{
    Set_State(STATE_POSITION, vPosition);
}
  
```



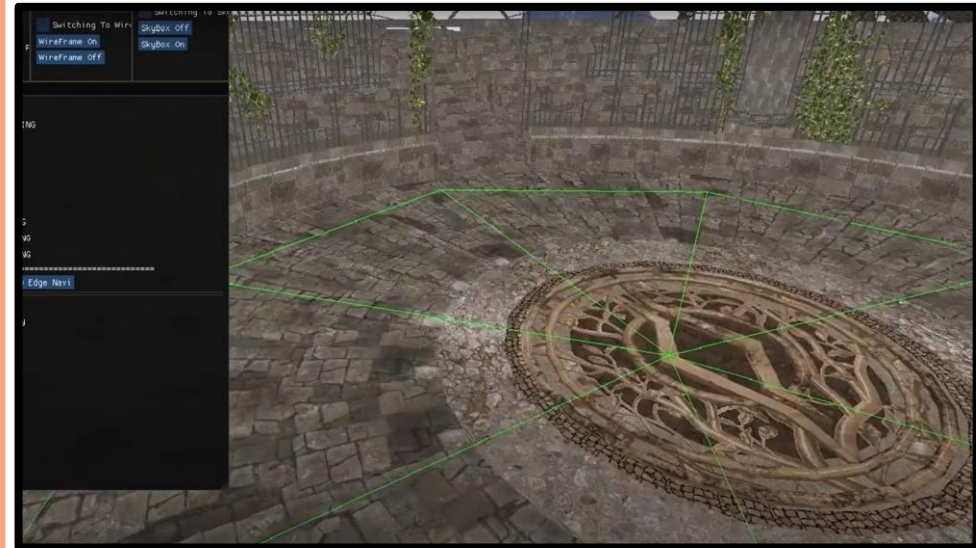

1-3

결과 및 최적화 기여 부분

- ◆ Navigation Tool을 통해 맵 에디터 상에서 직접 이동 가능 영역(Cell)을 생성 및 수정할 수 있게 되었습니다.
- ◆ 삼각형 Cell 기반의 Navigation Mesh를 구축하여, 플레이어 및 몬스터가 불필요한 충돌 없이 자연스럽게 이동할 수 있습니다.
- ◆ Edge Navigation(슬라이딩 벡터)을 적용하여, 경계선 충돌 시에도 부드럽게 벽을 따라 미끄러지는 이동이 가능합니다.
- ◆ Navigation 데이터는 Binary 형태로 저장되어, 로딩 시 빠른 접근과 각 스테이지마다 별개로 활용됩니다.



- ◆ 충돌 검사 연산 회피
 - 이동 가능한 영역을 Navigation - Cell 기반으로 한정하여, 불필요한 지형 오브젝트 간 충돌 검사 연산을 대폭 감소시켰습니다.
- ◆ 엔진 구조 통합
 - Navigation 기능을 Component 기반으로 추상화하여 모든 GameObject에 재사용이 가능합니다.
- ◆ 디버그 & 시각화 기능
 - Editor 상에서 Navigation 영역을 실시간 수정·디버깅할 수 있도록 지원하여 개발 생산성과 유지보수성을 크게 향상시켰습니다.





2-1

슬라이딩 벡터 구현 & Edge Navigation의 필요성

◆ 슬라이딩 벡터 도입

- 기존의 네비게이션 시스템은 Cell 내부 이동까지만 처리해주었기 때문에, Cell 들이 만나는 경계선에 충돌하면 움직임이 멈추었습니다. 이를 해결하기 위해 슬라이딩 벡터를 구현하게 됩니다.

```
for (_uint i = 0; i < CCell::LINE_END; ++i)
{
    // 방향 벡터 구하기 (vPosition = 플레이어의 월드 위치)
    _vector vDir = (vPosition - Get_Point(i));

    // Cell의 01 라인?, 12 라인?, 20 라인?.
    _vector vLine = (Get_Point((i + 1) % 3) - Get_Point(i));

    // 법선 벡터 구하기 (xz 평면 기준으로 수직인 법선 벡터)
    _vector vNormal = Normalize(Set(Z * -1, 0, X, 0));

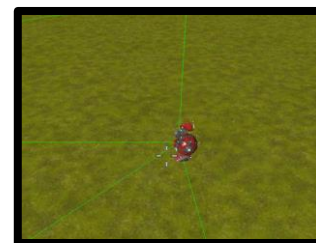
    // 방향벡터와 법선벡터 내적→음수이면 Cell 내부, 양수이면 바깥.
    if (0.0f < XMVectorGetX(XMVector3Dot(vDir, vNormal)))
    {
        // 투영벡터 플레이어 위치에서 충돌할 방향벡터.
        _vector vLineCollisionPoint = Get_Point(i) +
            Dot(vDir, vLine) * vLine / Dot(vLine, vLine);

        _vector vSlideDir = Normalize(vLineCollisionPoint
            - vPosition);

        return vPosition + (vSlideDir * 0.1f);
    }
}
```

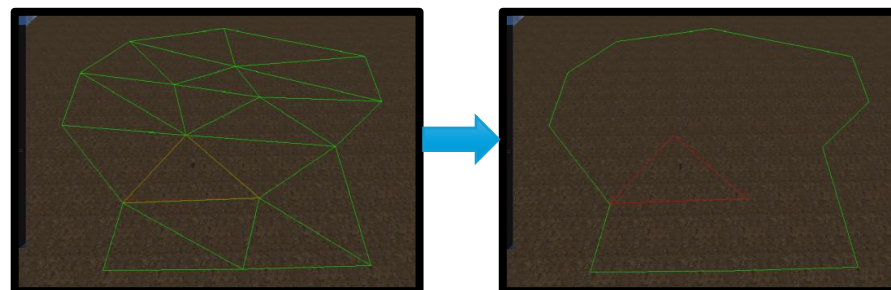
◆ 슬라이딩 벡터의 한계

- 3개 이상의 Cell이 한 꼭짓점에서 만나는 경우에는 슬라이딩 방향 판단이 모호해져 이동이 불안정해졌습니다.



◆ Edge Navigation 도입

- 외곽 Edge 만을 기준으로 이동을 제어하여 꼭짓점 부근에서의 충돌 모호성을 제거합니다.
- 슬라이딩 벡터만으로 해결되지 않던 경계 예외처리를 보완합니다.





2-2

Edge Navigation 구현과정

◆ Edge 라인 네비 셀 만들기

- 이웃이 없는 경계선만 남기기

```
// Edge 라인 네비 셀 만들기
// 이웃이 없는 경계선만 남기기
for (auto& pCell : m_Cells) {
    for (int i = 0; i < 3; ++i) {
        // 현재 Edge가 다른 Cell과 맞닿아 있으면 내부선 → 패스
        if (pCell->Get_NeighborIndex((CCell::LINE)i) != -1)
            continue;

        // 현재 Cell의 i번째 점(A)과 (i+1)%3번째 점(B)을
        // Edge의 두 점으로 설정
        _float3 v0, v1;
        // Edge 시작점 (A)
        XMStoreFloat3(&v0, pCell->Get_Point((CCell::POINT)i));

        // Edge 끝점 (B)
        XMStoreFloat3(&v1, pCell->Get_Point((CCell::POINT)
            ((i + 1) % 3)));

        // A-B를 잇는 한 변(Edge)
        _float3 line[2] = { v0, v1 };

        // 외곽선만 렌더링용 Edge 셀로 생성
        CEdge_Cell* pEdgeCell = CEdge_Cell::Create(m_pDevice,
            m_pContext, line);
        m_RenderEdgeLine.push_back(pEdgeCell);}}

```

◆ CEdge_Cell + CEdgeCell_Buffer 역할

- 실제로 선분 형태로 그려줍니다.

```
m_ePrimitiveTopology = D3D11_PRIMITIVE_TOPOLOGY_LINELIST;
```

◆ 최종 그려주기

- _iRenderTypeIndex == 0 -> 기존 Cell 네비게이션 그리기 (면 전체)
- _iRenderTypeIndex == 1 -> 외곽 Edge만을 렌더링

```
if (_iRenderTypeIndex == 0)
{
    for (auto& pCell : m_Cells)
        pCell->Render();
}

else if (_iRenderTypeIndex == 1)
{
    for (auto& pEdge_Cell : m_RenderEdgeLine)
        pEdge_Cell->Render();
}

```





2-3

Edge Navigation 기반 슬라이딩 벡터

◆ 보완된 슬라이딩 벡터 구현 코드 (Setting_SlidingMove 함수)

```

if (m_iNeighborIndex == -1) // 인접 Cell이 없으면 외곽 Edge 충돌
{
    // vPosition = 플레이어의 월드 위치
    for (i = 0; i < 3; ++i)
    {
        // Edge 방향 벡터
        _vector vLine =
            m_EdgeCells[m_iCurrentCellIndex].LINE_EDGE_1 + i;

        // Edge 법선 (xz 평면 기준)
        _vector vNormal = Normalize(Set(Z * -1, 0, X, 0));

        // 플레이어 위치 기준으로 Edge까지의 거리 계산
        _vector vCross = Cross(vLine,
                               (_vWorldPos - Get_Point(i)));
        _float fDist = Length(vCross) / Length(vLine);

        // 거리(key), 법선(value)로 저장
        m_mapSlide.emplace(fDist, vNormal);
    }
    auto& iter = m_mapSlide.begin();
    // 가장 가까운 Edge의 법선 선택
    _vector vLineNormal = Normalize(iter->second);

    // 슬라이딩 벡터 계산 (법선 성분 제거)
    _vector vDir = Normalize(_vLook);
    _vector vSlide = vDir -
        Dot(vDir, vLineNormal) * vLineNormal;

    return vSlide;
}

```

◆ 반환된 슬라이딩 방향 벡터를 각 GameObject의 이동 로직을 담당하는 Transform 컴포넌트에서 사용합니다. (Go_Straight 함수)

```

if (!pNavigation->isMove(vPosition))
{
    // 해당 방향으로 갈 수 있게끔 위치에 vDir을 더한다
    _vector vDir = pNavigation->Setting_SlidingMove
        (vPosition, vLook);

    vPosition -= XMVector3Normalize(vLook) * m_fSpeedPerSec;
    vPosition += XMVector3Normalize(vDir) * m_fSpeedPerSec;
}

```

◆ 최종적으로 GameObject(플레이어 & 몬스터)들은 부착된 Transform 컴포넌트의 이동 함수를 호출해, 보완된 네비게이션 영역을 움직일 수 있게 됩니다.

```

m_pTransformCom->Go_Straight(fTimeDelta * 2.f,
                             pObject->m_pNavigationCom);

```





3-1

픽셀 피킹의 필요성

◆ 일반 피킹(지형 피킹 & 메쉬 피킹)의 한계

- 기존에는 지형 피킹과 메쉬 피킹 두 가지 버전으로 사용해서 오브젝트 배치 등의 작업을 진행했습니다.
- 하지만 메쉬 피킹 같은 경우, 클릭 한 번으로 오브젝트를 선택할 때 마다 매 프레임마다 수백~수천 개의 폴리곤 검사를 수행하여 FPS 저하, 디버깅 불편, 작업 속도 지연 등의 문제가 발생했습니다.
- 이를 해결하기 위해 CPU 중심의 피킹 연산을 GPU 기반으로 옮겨 빠른 클릭 선택 시스템 구현이 필요했습니다.

```
for (size_t j = 0; j < VTXCNT * 3; ) {
    if (TriangleTests::Intersects(fvRayPos, fvRayDir,
        XMLoadFloat3(&pMeshTexPos[pIndices[j++])),
        XMLoadFloat3(&pMeshTexPos[pIndices[j++])),
        XMLoadFloat3(&pMeshTexPos[pIndices[j++])), fDist))
    {
        _vector vHit = fvRayPos + (fDist * fvRayDir);
        vHit = XMVector3TransformCoord(vHit,
            XMMatrixInverse(nullptr, matWorld));
        m_Distance.push_back({ fDist, vHit });
    }
}
```

- 모든 폴리곤을 순회하며 Ray - Triangle 교차 여부를 검사하고 교차된 지점을 월드 좌표로 변환하여 저장하기 때문에 CPU의 부담이 커지게 됩니다.

- 기존 코드에서는 오브젝트가 다음과 같이 겹쳐 있을 경우, Ray가 앞 오브젝트의 면도 뚫고, 뒤 오브젝트까지 닿을 수 있다는 문제 또한 있었습니다.
- 따라서 교차된 값들 에서 가장 작은 값을 선택했으나, 오브젝트가 얇거나 폴리곤 간격이 좁은 경우에도 한계가 있었습니다.
- 게다가 모든 과정이 CPU 연산이기 때문에 과정이 빠르지 않았습니다.



- 목적 대상 오브젝트를 뚫고 뒤에 겹친 오브젝트 폴리곤의 면이 피킹 되는 현상을 와이어프레임 모드로 디버깅하여 확인했습니다.



3-2

픽셀 피킹 구현과정 (1)

◆ 픽셀 깊이 전용 렌더타겟 생성

- 화면 픽셀의 깊이값을 담은 전용 렌더타겟을 생성합니다.

```
m_pGameInstance->Add_RenderTarget(L"Target_PickDepth",
                                   width, height,
                                   DXGI_FORMAT_R32G32B32A32_FLOAT, _float4(0, 0, 0, 0));

m_pGameInstance->Add_MRT(L"MRT_GameObjects",
                        L"Target_PickDepth");
```

◆ GPU에서 그려진 픽셀 데이터를 CPU가 읽을 수 있게끔 텍스처를 생성합니다.

```
TextureDesc.Usage = D3D11_USAGE_STAGING;
TextureDesc.BindFlags = 0;
TextureDesc.CPUAccessFlags = D3D11_CPU_ACCESS_READ;
TextureDesc.MiscFlags = 0;

m_pDevice->CreateTexture2D(&TextureDesc, nullptr,
                          &m_pTexture2D));
```

◆ 셰이더에서 픽셀 깊이 정보를 출력합니다.

- 픽셀 단위로 z/w 깊이 정보를 Target_PickDepth에 렌더링 합니다.

```
Out.vPickDepth = vector In.vProjPos.z / In.vProjPos.w,
                  In.vProjPos.w 1.f 0.f
```

◆ 렌더링 된 PickDepth 텍스처를 CPU 접근용 텍스처에 복사합니다.

```
Copy_RenderTarget_TextureResource(TEXT("Target_PickDepth"),
                                   m_pTexture2D);
```

◆ 마우스 좌표를 스크린에서 클라이언트 좌표로 변환 후, 해당 픽셀의 값을 Map을 통해 읽어옵니다.

```
m_pContext->Map(m_pTexture2D, 0, D3D11_MAP_READ_WRITE, 0,
               &SubResource);

_uint iPixelIndex = (_uint)(ptMouse.y * m_fViewportWidth
                             + ptMouse.x);

_float4 vDepthDesc = static_cast<_float4*>
                     (SubResource.pData)[iPixelIndex];

m_pContext->Unmap(m_pTexture2D, 0);
```

◆ 픽셀의 깊이와 뷰/투영 역행렬을 이용해, 화면 픽셀을 실제 월드상의 3D 좌표로 복원합니다.

```
vWorldPos = XMVector3TransformCoord(vWorldPos,
                                     Get_Transform_Matrix_Inverse(CPipeLine::D3DTS_PROJ));

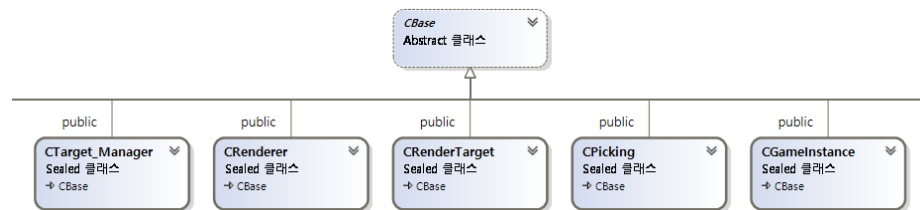
vWorldPos = XMVector3TransformCoord(vWorldPos,
                                     Get_Transform_Matrix_Inverse(CPipeLine::D3DTS_VIEW));

XMStoreFloat3(_pOut, vWorldPos);
```



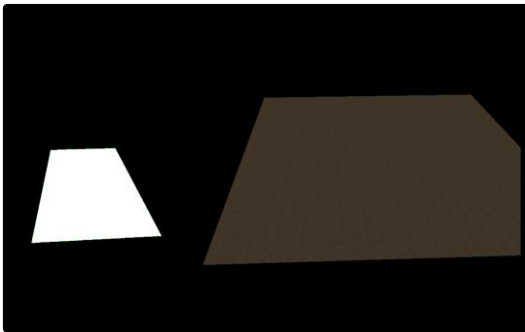
3-2

픽셀 피킹 구현과정 (2)

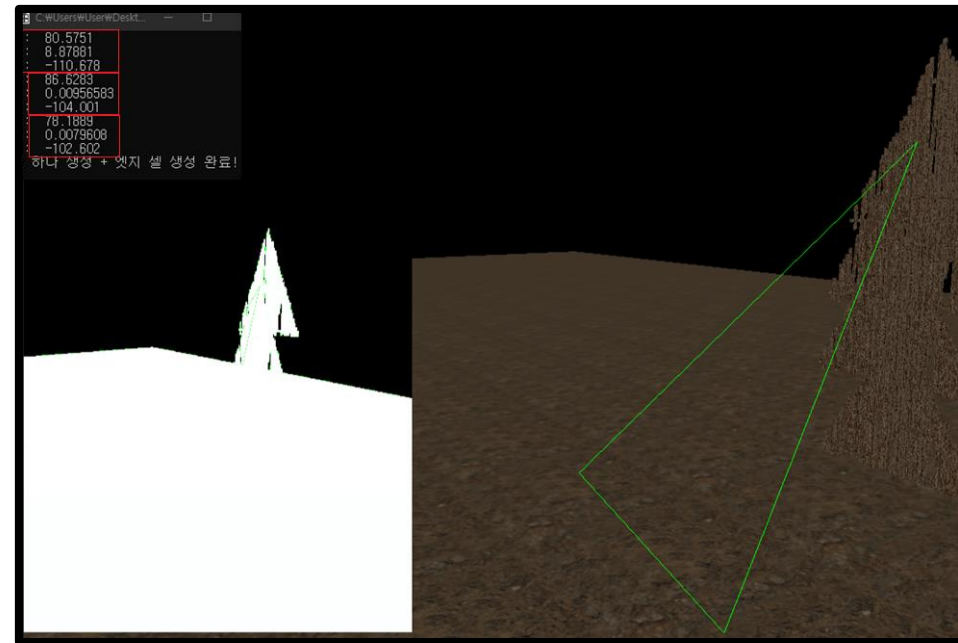


◆ 구조 요약

- 결국 픽셀 피킹은 화면의 각 픽셀 깊이값을 따로 담기 위한 전용 렌더타겟을 생성하고, 여기에 연결된 전용 텍스처에 픽셀별로 z/w 값이 정보를 셰이더 단계에서 렌더링하는 구조입니다.



- Terrain 오브젝트의 픽셀 셰이더가 실행될 때 깊이값을 채워 그리는데 새로운 텍스처를 전용 렌더타겟에 보여주고 있습니다.
- GPU에서 그려진 깊이 텍스처를 CPU가 읽을 수 있는 텍스처로 복사해 다시 그려주는 것입니다.



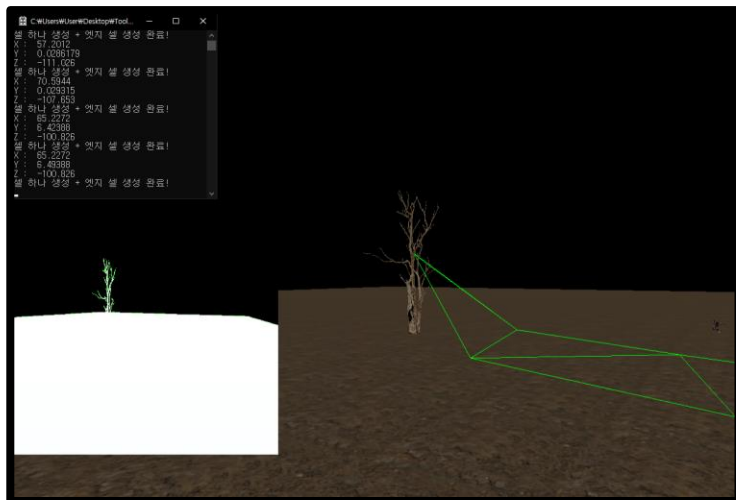
◆ 최종 피킹 연산

- 사용자가 마우스로 클릭한 화면 좌표의 픽셀 값을 읽어 NDC → 월드 좌표로 역변환하여 실제 3D 공간상의 클릭 지점을 얻어냅니다.
- 즉, 최종적으로 디버그 화면에 보이는 Target_PickDepth의 픽셀 하나하나에는 깊이 정보가 기록되어 있기 때문에, 해당 영역 안에서 보이는 모든 픽셀은 피킹이 가능한 대상이 됩니다.

3-3

결과 및 최적화 기여

◆ 오브젝트를 그려주는 각 셰이더에서 깊이값을 출력해주면 피킹이 가능해지기 때문에 이를 활용해 훨씬 수월한 작업을 진행할 수 있습니다.



◆ 가장 큰 목적이었던 정교한 네비게이션 메쉬 영역을 빠르게 지정할 수 있게 됩니다. 정확한 세 점을 찍어 어디든 Cell을 생성할 수 있게 되었습니다.

◆ 메쉬 피킹을 하지 않고, 복잡한 모델에서도 피킹이 가능해집니다.

- 수 천개의 폴리곤 & Ray 교차 연산에서 GPU 한 번의 드로우콜로 대체하여 프레임 드랍현상 제거, 작업 속도를 향상시켰습니다.

◆ 픽셀 피킹을 응용해, 총알이 벽에 부딪혀 탄흔을 남기는 이펙트 또한 구현이 가능해졌습니다.



```
_float3 vHitPos;
if (m_pGameInstance->Compute_PickPos(&vHitPos))
{
    pImpactEffect->Activate_ImpactEffect(vHitPos);
}
DeActivate_Bullet();
```

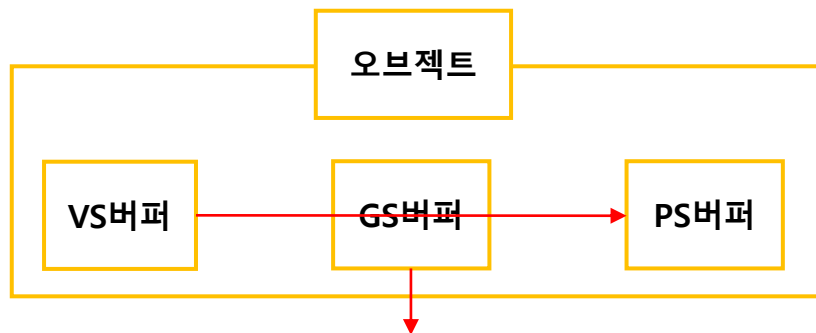



4-1

극적인 연출의 필요성과 구조

◆ 작업 중 만들어지는 게임의 퀄리티를 더욱 높이기 위해서 화려한 연출이 큰 역할을 한다고 생각했습니다.

- 전투나 보스 연출과 같은 상황에서는 단순 모델 출력만으로 시각적인 임팩트가 부족하다가 판단했습니다..
- 기존의 방식으론 모델의 형태 변화, 폭발 같은 효과를 표현하기가 어려웠기 때문에 Geometry Shader를 활용하게 되었습니다.
- 자주 발생하는 이펙트인 만큼, GPU에서 연산을 수행할 필요가 있습니다.

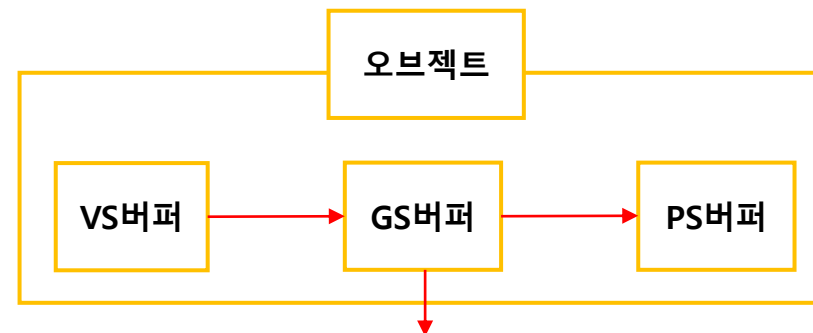


◆ 버텍스 버퍼에서 픽셀 버퍼로 진행하며 렌더링을 해주는 과정에서 GS 버퍼가 추가됩니다.

- 변화를 일으킬 트리거가 작동하기 전에는 GS버퍼는 0으로 동작하여 VS버퍼에서 GS버퍼의 흐름을 유지하고 있습니다.

◆ 버텍스 버퍼 -> GS 버퍼 -> 픽셀 버퍼의 렌더링 파이프라인

- 플레이어 혹은 특정 대상과의 충돌을 감지합니다.
- 충돌 후 시간에 따라 오브젝트들의 변수 변화량 값을 증가시킵니다. (폭발량, 낙하량, 회전값 등)
- 위 값들을 GeoMetry Shader 로 바인딩합니다. (VS버퍼 -> GS버퍼 -> PS버퍼 가동 시작)
- 이제 GeoMetry Shader 에서 정점별로 특정 연산을 수행합니다.



◆ GS에서 특정 연산을 수행하고 난 결과물을 PS에서 이어받아 렌더링 및 디졸브 텍스처 샘플링을 담당합니다.



4-2

동적인 연출을 위한 수학, 물리 공식 적용 (1)

◆ 구현한 연출은 다음과 같습니다.

- 부서지는 오브젝트 연출 (상자, 울타리 오브젝트)
- 검투사 보스 공격 시, '대검'과 지형이 부딪힐 때 튀어나오는 돌맹이 파편 연출
- 박쥐 보스 패턴 중 소환하는 자수정 파편화 연출



◆ 정점을 '분할->이동->회전->낙하->Dissolve 처리'으로 연산을 수행합니다.

◆ 회전 처리에서의 문제점

- 분할된 파편마다 회전축이 다르기에 기존 X Y Z 고정 회�행렬로는 자연스러운 회전이 불가능했습니다.
- 이를 해결하기 위해 로드리게스 회전 행렬 공식을 적용했습니다.

$$\begin{aligned}
 R_{\vec{\omega}}(\theta) &= e^{\vec{\omega} \theta} \\
 &= I + \vec{\omega} \sin \theta + \vec{\omega}^2 (1 - \cos \theta) \\
 &= \begin{bmatrix} \cos \theta + \omega_x^2 (1 - \cos \theta) & \omega_x \omega_y (1 - \cos \theta) - \omega_z \sin \theta & \omega_y \sin \theta + \omega_x \omega_z (1 - \cos \theta) \\ \omega_z \sin \theta + \omega_x \omega_y (1 - \cos \theta) & \cos \theta + \omega_y^2 (1 - \cos \theta) & -\omega_x \sin \theta + \omega_y \omega_z (1 - \cos \theta) \\ -\omega_y \sin \theta + \omega_x \omega_z (1 - \cos \theta) & \omega_x \sin \theta + \omega_y \omega_z (1 - \cos \theta) & \cos \theta + \omega_z^2 (1 - \cos \theta) \end{bmatrix}
 \end{aligned}$$

◆ 분할시킨 파편들 마다의 회전축과 시간 증가량에 따른 회전각으로 공식을 적용한 3x3 회�행렬을 생성했습니다.



4-2

동적인 연출을 위한 수학, 물리 공식 적용 (2)

◆ 메쉬(모델)의 분할 (이동)

- 모델의 중심 기준의 정점 위치로 구역을 분할합니다.
- 각 구역별 방향벡터를 설정해줍니다.

◆ 폭발값, 중력 공식을 적용해줍니다.

```
// V0 * t + (0.5) * g * t^2
float fFallingValue = (fInitialPopVelocity * fFallingTime)
+ (0.4f) * (fGravityPower * fFallingTime * fFallingTime);
vMoveValue.y += fFallingValue;
```

◆ 로드리게스 회전행렬을 계산합니다.

```
float3x3 matRotation =
{
    fCosAngle + ((fRotationX * fRotationX) * (1 - fCosAngle)),
    ((1 - fCosAngle) * fRotationX * fRotationY) - (fSinAngle * fRotationZ),
    ((1 - fCosAngle) * fRotationX * fRotationZ) + (fSinAngle * fRotationY),

    ((1 - fCosAngle) * fRotationX * fRotationY) + (fSinAngle * fRotationZ),
    fCosAngle + (fRotationY * fRotationY * (1 - fCosAngle)),
    (-fRotationX * fSinAngle) + (fRotationY * fRotationZ * (1 - fCosAngle)),

    ((1 - fCosAngle) * fRotationX * fRotationZ) - (fRotationY * fSinAngle),
    (fRotationX * fSinAngle) + (fRotationY * fRotationZ * (1 - fCosAngle)),
    fCosAngle + ((fRotationZ * fRotationZ) * (1 - fCosAngle))
};
```

◆ 정점별 이동과 회전 결과를 적용합니다.

```
for (int i = 0; i < 3; ++i)
{
    float fVerticesYPos = input[0].vWorldPos.y + vMoveValue.y;
    float3 vResultMoveValue = vMoveValue;

    if (fVerticesYPos ≤ g_ModelPosition.y)
    {
        float fCurrentVerticesYpos = input[0].vWorldPos.y;
        vResultMoveValue.y = g_ModelPosition.y - fCurrentVerticesYpos;
    }

    bool bIsDissolve = ((input[i].vWorldPos.y + vResultMoveValue.y)
    ≤ g_ModelPosition.y);

    Out.fDissolve = bIsDissolve ? 1.0f : 0.0f;

    float3 vVerticesPosOffset = input[i].vWorldPos.xyz - vPartCenter;
    float3 vRotationOffset = mul(vVerticesPosOffset, matRotation);
    float3 vResultVerticesPos = vPartCenter + vRotationOffset
    + vResultMoveValue;

    Out.vWorldPos = float4(vResultVerticesPos, 1.0f);
    Out.vPosition = mul(mul(Out.vWorldPos, g_ViewMatrix), g_ProjMatrix);

    triStream.Append(Out);
}
```

◆ 각각의 방향 벡터로 이동, 중력 적용, 회전 적용의 결과로 인해 최종적으로 분해되면서 자연스럽게 날아가는 연출이 완성됩니다.



4-3

모델의 정점 정보 수정

◆ Geometry Shader를 이용한 정점 정보 수정

- 모델의 정점 좌표를 GS버퍼에서 직접 조정하여 실시간 변형이 가능하도록 구현했습니다.
- GS에서 시간에 따라 Sin 파동을 생성해, 각 정점을 Normal 방향으로 미세하게 진동시킵니다.
- 정점의 월드 위치를 기준으로 위상 차이를 주어 모델 전체가 자연스럽게 흔들리게끔 구현했습니다.
- 모델의 애니메이션 파일 에셋 없이 실시간으로 Idle 애니메이션을 표현해 볼 수 있었습니다.



- Idle 시에 자체적으로 움직이다가 충돌 시, 마찬가지로 분해됩니다.

```
for (int i = 0; i < 3; ++i)
{
    GS_OUT_DESTRUCT Out = (GS_OUT_DESTRUCT) 0;
    float3 vWorldPos = input[i].vWorldPos.xyz;
    float3 vLocalOffset = vWorldPos - g_ModelPosition.xyz;

    float fFarOffsetValue =
        (vLocalOffset.x + vLocalOffset.z) * 2.0f;

    float fWiggleValue =
        sin(fWiggleTime * 10.0f + fFarOffsetValue) * 0.02f;

    vLocalOffset += input[i].vNormal.xyz * fWiggleValue;

    float3 vFinalPos = g_ModelPosition.xyz
        + vLocalOffset;

    Out.vWorldPos = float4(vFinalPos, 1);
    Out.vPosition = mul(Out.vWorldPos,
        mul(g_ViewMatrix, g_ProjMatrix));
    Out.vProjPos = input[i].vProjPos;
    triStream.Append(Out);
}
triStream.RestartStrip();
```

4-4

동일 메쉬 복제

◆ 하나의 오브젝트를 여러 개로 복제하여, 돌맹이가 튀는 연출을 구현했습니다.

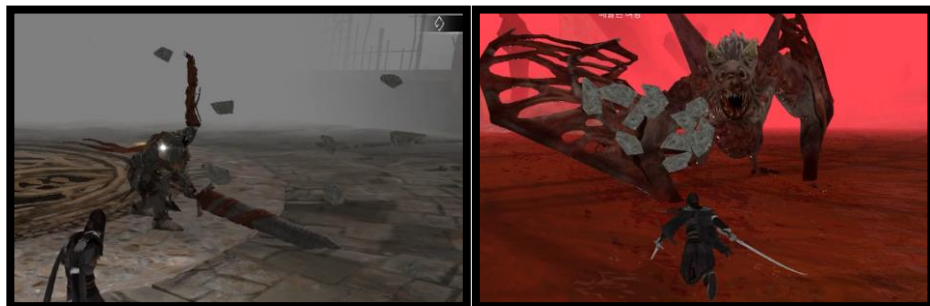
◆ VS에서 원본 돌맹이 한 개의 정점만 넘겨주고, GS가 이를 넘겨 받아, 여러 개로 복제해서 각자의 방향으로 튕기게끔 합니다.

◆ 최종적으로 PS에서는 각 복제된 메쉬를 독립적으로 그리게 됩니다.

◆ 이전 까지는 파편화 라는 연출로, 기존 삼각형을 쪼개서 늘렸다면, 이번에는 삼각형 전체를 통째로 여러 번 그리는 구조입니다.

- GS의 입력 매개변수에 `input[3]`, 즉 삼각형 1개를 12번 복제
- 위의 과정을 원본의 폴리곤 개수만큼 반복

◆ 마찬가지로 회전행렬 + 중력이 적용되어 복제된 돌맹이가 각 방향으로 자연스럽게 날아가는 연출이 완성됩니다.



```
#define PARTS_COUNT 12
[maxvertexcount(3 * PARTS_COUNT)]
void GS_MAIN_MAGNITUDE(triangle GS_IN_DESTRUCT input[3],
    inout TriangleStream<GS_OUT_DESTRUCT> triStream)
{
    for (int iPartIndex = 0; iPartIndex < PARTS_COUNT; ++iPartIndex)
    {
        // 각 파트마다 방향벡터 + 폭발값 + 중력 + 회전행렬 생성.

        // 이제 모델의 정점 [0] [1] [2] 3개를 픽셀 셰이더로 보낼 준비.
        for (int i = 0; i < 3; ++i)
        {
            GS_OUT_DESTRUCT Out = (GS_OUT_DESTRUCT) 0;

            Out.vTexcoord = input[i].vTexcoord;
            Out.vNormal = input[i].vNormal;
            Out.vTangent = input[i].vTangent;
            Out.vBinormal = input[i].vBinormal;

            float3 vLocalOffset = input[i].vWorldPos.xyz
                - g_ModelPosition.xyz;

            float3 vRotated = mul(vLocalOffset, matRotation);

            float3 vFinalPos = g_ModelPosition.xyz
                + vRotated + vMoveValue;

            Out.vWorldPos = float4(vFinalPos, 1);
            Out.vPosition = mul(Out.vWorldPos,
                mul(g_ViewMatrix, g_ProjMatrix));

            triStream.Append(Out);
        }

        // 반복분 1회 수행 후, 다음 파트 수행 준비 (PARTS_COUNT - 1번)
        triStream.RestartStrip();
    }
}
```




4-5

결과 및 기여 부분

◆ Geometry Shader를 활용하여 다양한 연출을 구현할 수 있었습니다.

- 기존에는 CPU에서 회전 등의 연산을 직접 수행했으나 이번에는 GPU 상에서 정점 단위 연산을 처리하도록 구조를 옮겼습니다.
- 그 결과, CPU 부하를 최소화한 채로 복잡한 폭발, 진동, 복제 등의 다양한 연출을 실시간으로 구현할 수 있었습니다.

◆ GPU 연산을 통해 성능이 크게 향상 되었습니다.

- GS 단계에서 정점, 삼각형 단위 처리를 수행하므로, CPU 개입 없이도 수십~수백 개의 오브젝트를 동시에 처리 가능했습니다.
- 특히 반복 연산이 많았던 복제, 파편화 연출 경우 큰 이득을 보았습니다.

◆ 개인 기여 및 배운 점

- Geometry Shader의 입력, 출력 구조를 설계하고, 더 자연스러운 회전을 위해 문서를 참고해 회전행렬 구현과, 중력 가속도와 같은 물리 수식을 적용할 수 있었습니다.
- VS -> GS -> PS 전체 파이프라인의 흐름을 이해하고, 제어해 볼 수 있었습니다.
- 프레임 저하 없이 자연스럽게 자연스러운 연출이 가능해졌습니다.

◆ 참고했던 문서, 블로그 링크

- 로드리게스 회전 행렬
<https://mathworld.wolfram.com/RodriguesRotationFormula.html>
- <https://luv-n-interest.tistory.com/754>
- Geometry Shader 문법
<https://learn.microsoft.com/en-us>



5-1

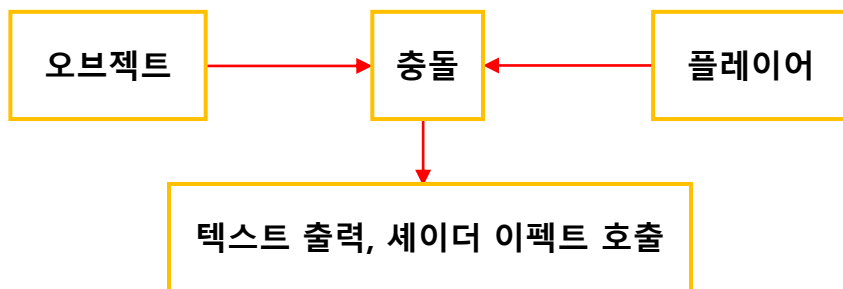
상호작용 시각화의 필요성

◆ 게임 중에 플레이어는 특정 오브젝트, NPC 등과 상호작용을 해야 합니다.

따라서, 플레이어가 어느 오브젝트와 충돌하여, 다음 로직이 수행되는지를 유저들이 인지할 수 있는 시각화 작업이 필요했습니다.

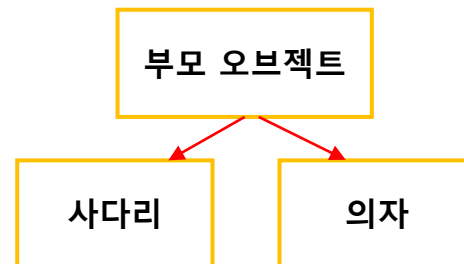
◆ 오브젝트와 상호작용 중이라는 시각화 작업 구현들은 다음과 같습니다.

- 오브젝트의 목적에 맞는 특정 UI Text 출력
- 계속 충돌 중임을 확인할 수 있는 Rim-Light 셰이더 구현



◆ 상호작용이 오브젝트들은 공통적으로 월드 좌표 -> HUD 좌표 계 변환이 적용된 UI Text 객체를 가지고 있습니다.

- 상호작용 오브젝트의 부모클래스가 생성될 때 UI Text객체의 포인터를 멤버로 저장 후, 자식 오브젝트들에게 공유합니다.



◆ 충돌하게 하면 Rim-Light 이펙트와 해당 UI가 렌더링 되어 각자 다음과 같은 상호작용이 가능해집니다.

- 사다리 : 맵 상에서 층간 이동이 가능
- 엘리베이터 : Fade In Out 후, 층간 이동이 가능
- 의자 : 플레이어 체력 회복 및 리스폰 위치 저장 가능
- NPC : 다른 스테이지로 이동 가능
- 게이트 : 특정 조건 만족 시, 오픈
- 아이템 : 드롭된 아이템 획득 가능



5-2

상호작용 UI Text

◆ 다음과 같은 좌표계 변환을 거친 UI Text 객체가 상호작용 오브젝트에서 호출되어 렌더링 됩니다.

```
// 월드 좌표 → 뷰 좌표 변환
_vector vViewPos = XMVector3TransformCoord(vWorldPos,
    XMLoadFloat4x4(&matView));

// 뷰 좌표 → 투영 좌표 변환
_vector vProjPos = XMVector3TransformCoord(vViewPos,
    XMLoadFloat4x4(&matProj));

// ( -1 ~ 1 ) 정규화 좌표 → ( 0 ~ 1 ) 스크린 좌표 변환
vScreenPos.x = (XMVectorGetX(vProjPos) + 1.0f) * 0.5f;
vScreenPos.y = (1.0f - XMVectorGetY(vProjPos)) * 0.5f;

// 뷰포트 해상도 적용
vScreenPos.x *= vViewportSize.x;
vScreenPos.y *= vViewportSize.y;

// 좌표계 중심 이동
vScreenPos.x -= vViewportSize.x * 0.5f;
vScreenPos.y = vViewportSize.y * 0.5f - vScreenPos.y;

// 최종 UI 버튼 위치 적용
m_pTransformCom->Set_State(CTransform::STATE_POSITION,
    XMVectorSet(m_fButtonPosition.x,
        m_fButtonPosition.y, 0.0f, 1.0f));
```

◆ 상호작용 전에는 활성화되지 않으며, 상호작용 후에는 DirectXTK의 SpriteFont 라이브러리가 텍스트 출력을 하게 됩니다.

```
m_pBatch->Begin(SpriteSortMode_Deferred);

    m_pFont->DrawString(m_pBatch,
        pText, vPosition, XMLoadFloat4(&vColor),
        fRotation, vOrigin, fScale,
        effects, layerDepth);

m_pBatch->End();
```

◆ 각 오브젝트들은 다음과 같은 형식으로 Text를 호출합니다.

```
_float4 vChairPosition;
XMStoreFloat4(&vChairPosition, vChairPos);

m_pButton->Set_WorldPosition(vChairPosition);
m_pButton->Set_ButtonText(TEXT("E"), TEXT("앉기"));
m_pButton->Activate_Button(true);
```





5-3

Rim Light 셰이더

◆ UI Text 외에도 Rim Light 셰이더를 적용해, 픽셀의 알파값에 페이딩 인아웃 효과를 추가함으로써 더욱 눈에 띄는 하이라이트 연출을 구현했습니다.

```
if (FAILED(m_pShaderCom->Bind_RawValue("g_fObjectAlpha",
    &m_fAlphaValue, sizeof(_float))))
    return E_FAIL;

m_pShaderCom->Begin(11);

m_pModelCom->Render(i);
```

◆ 시간 변화량 변수를 바인딩 해주어, Rim Light가 서서히 밝아지고, 알아지게 했습니다.



◆ Rim Light 셰이더 처리 과정

- 카메라에서 현재 픽셀로 가는 방향 벡터를 계산합니다.
- 픽셀의 노말 방향과 카메라 방향의 내적을 이용해, 윤곽 부분일 수록 값이 커지도록 계산합니다.
- 최종 색상과 세기, 알파값들을 조합해 최종 빛 색상을 계산합니다.
- 마지막으로 기존 픽셀 색상에 연산한 빛 색상을 추가합니다.

```
PS_OUT_RIMLIGHT Out = (PS_OUT_RIMLIGHT) 0;

float3 vDirection = normalize(g_vCamPosition.xyz
    - In.vWorldPos.xyz);

float fRim = (1 - dot(vDirection, In.vNormal.xyz));
fRim = pow(abs(fRim), g_RimPower);

float4 vRimLight = fRim * fRimStrength
    * g_RimColor * g_fObjectAlpha;
Out.vColor += vRimLight;

return Out;
```

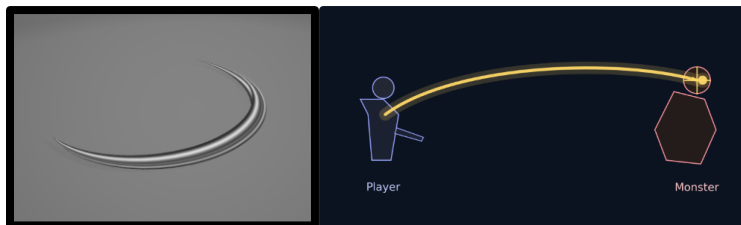


5-4

몬스터와 빔-라인 연결 (조준선)

- ◆ 락온 상태일 때, 시각화 할 수 있는 빔-라인을 구현했습니다.
- ◆ 빔-라인을 구현하기 위해 선 처럼 보이는 메쉬를 이용했습니다.
 - 플레이어 위치를 시작점, 락온된 몬스터의 특정 bone 좌표를 끝점으로 지정합니다.
 - 그 두 점을 이용해 선 메쉬의 진행 방향과 실제 길이를 구해줍니다.
 - 선 메쉬의 기준 위치를 이 두 점의 중앙으로 설정해 양쪽을 잇는 중심선으로 배치되게 했습니다.

```
vEndPos = XMVectorSetY(vEndPos, XMVectorGetY(vEndPos));
_vector vDir = vEndPos - XMVectorGetX(&m_vStartPos);
_float fLength = XMVectorGetX(XMVector3Length(vDir));
vDir = XMVector3Normalize(vDir);
_vector vCenter = (XMVectorGetX(&m_vStartPos) + vEndPos)
                  * 0.5f;
```



- ◆ 선 메쉬의 회전 방향을 제대로 플레이어 -> 몬스터 방향으로 맞추기 위해서, 축을 재정의합니다.

```
_vector vRight = vDir;
_vector vUp = XMVectorSet(0.f, 1.f, 0.f, 0.f);
_vector vLook = XMVector3Normalize(
    XMVector3Cross(vUp, vRight));
vUp = XMVector3Normalize(XMVector3Cross(vRight, vLook));
```

- ◆ 플레이어와 몬스터, 두 점 사이 거리에 맞춰서 선의 길이를 조절합니다.

```
m_pTransformCom->Scaling(_float3(0.01f * fLength,
    0.01f, 0.01f));
```

- ◆ 마지막으로 셰이더에서 선 메쉬에 맵핑된 텍스처의 UV 좌표 중 y좌표를 이동시켜 선이 플레이어에서 몬스터 쪽으로 조준선을 쏘는 듯한 연출을 구현했습니다.

```
float2 UV = In.vTexcoord;
UV.y += g_Time * 0.7f;
```





6-1

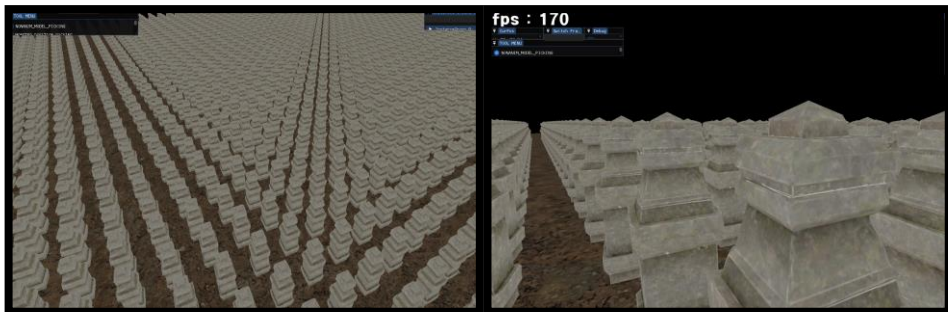
인스턴싱의 필요성과 렌더링 흐름

◆ 게임 스테이지를 제작할 때, 동일한 모델이 여러 번 배치되면 CPU가 각 오브젝트 마다 DrawCall을 개별적으로 호출해야 합니다.

◆ 수백~수천 개의 오브젝트가 존재할 경우, CPU가 그리라는 명령을 그만큼 내리기 때문에, 병목현상이 발생하고, 이는 프레임 저하로 이어졌습니다

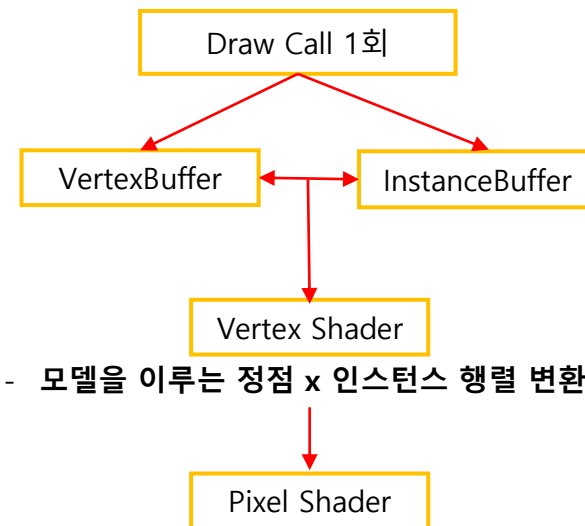
◆ 따라서 메쉬 인스턴싱을 적용해, 동일한 메쉬 데이터를 사용하는 오브젝트를 하나의 DrawCall로 묶어서 처리할 필요가 있었습니다.

- CPU는 "이 메쉬를 여러 위치/스케일/회전으로 렌더링하라" 라는 한번의 명령을 전달하고, GPU는 동일 정점을 서로 다른 월드행렬로 변환하며 반복 출력합니다.



◆ 인스턴싱 렌더링 흐름

- 1번의 DrawCall로 여러 개의 인스턴스 데이터를 GPU에 전달합니다.
- GPU는 정점 버퍼와 인스턴스 버퍼를 함께 바인딩 하여, 셰이더에서 각 인스턴스의 월드행렬을 적용해 반복 출력합니다.





6-2

상세 구조 (1)

◆ Input Layout 설정

- 슬롯을 2개를, 하나는 일반 정점 버퍼, 다른 하나는 인스턴스 버퍼로 설정합니다.
- GPU는 0번 슬롯은 정점, 1번 슬롯은 인스턴스 4x4행렬이라는 사실을 알게됩니다.

```
const D3D11_INPUT_ELEMENT_DESC
VTX_MODEL_INSTANCE::Elements[] = {

    { "POSITION", 0, DXGI_FORMAT_R32G32B32_FLOAT, 0, 0,
      D3D11_INPUT_PER_VERTEX_DATA, 0 },
    { "NORMAL", 0, DXGI_FORMAT_R32G32B32_FLOAT, 0, 12,
      D3D11_INPUT_PER_VERTEX_DATA, 0 },
    { "TEXCOORD", 0, DXGI_FORMAT_R32G32_FLOAT, 0, 24,
      D3D11_INPUT_PER_VERTEX_DATA, 0 },
    { "TANGENT", 0, DXGI_FORMAT_R32G32B32_FLOAT, 0, 32,
      D3D11_INPUT_PER_VERTEX_DATA, 0 },

    { "INSTANCE_MATRIX", 0, DXGI_FORMAT_R32G32B32A32_FLOAT, 1,
      0, D3D11_INPUT_PER_INSTANCE_DATA, 1 },
    { "INSTANCE_MATRIX", 1, DXGI_FORMAT_R32G32B32A32_FLOAT, 1,
      16, D3D11_INPUT_PER_INSTANCE_DATA, 1 },
    { "INSTANCE_MATRIX", 2, DXGI_FORMAT_R32G32B32A32_FLOAT, 1,
      32, D3D11_INPUT_PER_INSTANCE_DATA, 1 },
    { "INSTANCE_MATRIX", 3, DXGI_FORMAT_R32G32B32A32_FLOAT, 1,
      48, D3D11_INPUT_PER_INSTANCE_DATA, 1 },
};
```

◆ 인스턴싱 오브젝트 초기화

- 인스턴스 데이터로 인스턴스 행렬을 계산합니다.

```
XMMATRIX matWorld = matScale *
                    matRotation * matPosition;

XMFLOAT4X4 tempMatrix;
XMStoreFloat4x4(&tempMatrix, matWorld);

instance.InstanceMatrix[0] =
    XMFLOAT4(tempMatrix._11, tempMatrix._12,
              tempMatrix._13, tempMatrix._14);

instance.InstanceMatrix[1] =
    XMFLOAT4(tempMatrix._21, tempMatrix._22,
              tempMatrix._23, tempMatrix._24);

instance.InstanceMatrix[2] =
    XMFLOAT4(tempMatrix._31, tempMatrix._32,
              tempMatrix._33, tempMatrix._34);

instance.InstanceMatrix[3] =
    XMFLOAT4(tempMatrix._41, tempMatrix._42,
              tempMatrix._43, tempMatrix._44);
```

- 인스턴스 행렬을 컨테이너에 저장합니다.

```
m_vecInstanceData.push_back(instance);
```

6-2

상세 구조 (2)

◆ GPU 인스턴스 버퍼 생성

- GPU용 인스턴스 버퍼를 생성합니다.
- CPU가 GPU에 “이 메쉬를 N개의 행렬로 그려라”라는 명령입니다.

```
m_pModelCom→Create_InstanceBuffer(m_iNumInstance,
                                   m_vecInstanceData.data()))
```

◆ 인스턴싱 오브젝트 렌더링

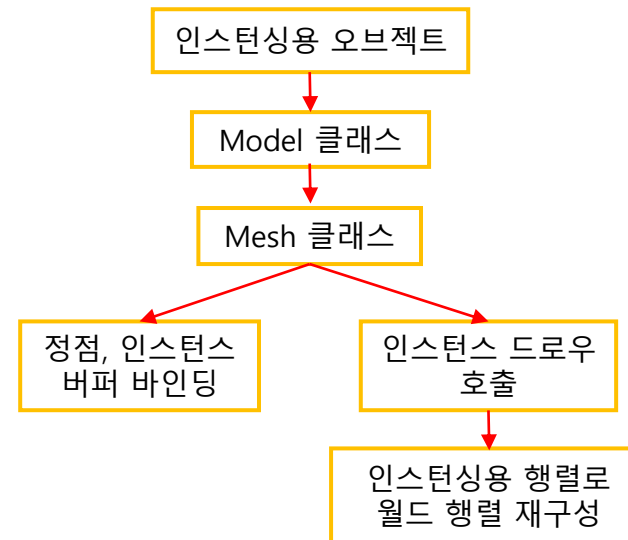
- 셰이더에 메테리얼과 텍스처 등을 바인딩 한 후, 셰이더를 시작하고, 인스턴싱 렌더를 호출합니다.

```
for (_uint i = 0; i < iNumMeshes; i++)
{
    if (FAILED(m_pModelCom→Bind_Material(m_pShaderCom, i,
    aiTextureType_DIFFUSE, "g_DiffuseTexture", 0)))
        return E_FAIL;
    if (FAILED(m_pModelCom→Bind_Material(m_pShaderCom, i,
    aiTextureType_NORMALS, "g_NormalTexture", 0)))
        return E_FAIL;
    m_pShaderCom→Begin(2);
    m_pModelCom→Render_Instance(i, m_iNumInstance);
}
```

◆ GPU 파이프라인 세팅

- Input Layout의 슬롯들이 GPU의 Input Assembler에 연결됩니다.
- 인스턴스 드로우 명령을 호출합니다.

```
HRESULT CModel::Render_Instance(_uint _iMeshIndex,
                                _uint _iNumInstanceNumber)
{
    m_Meshes[_iMeshIndex]→
        Bind_InputAssembler_Instance(m_pInstanceBuffer);
    m_Meshes[_iMeshIndex]→
        Render_Instance(m_pInstanceBuffer,
                        _iNumInstanceNumber);
}
```



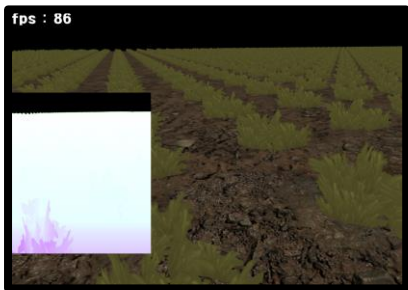


6-3

결과 및 기여

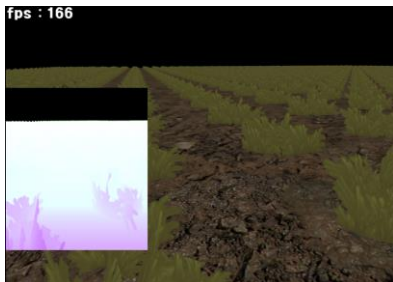
◆ 인스턴싱을 적용한 결과

- 기존 동일 모델(풀)을 드래그 형식으로 1600개를 배치



- Fps : 86 ~ 95

- 메쉬 인스턴싱으로 1600개를 동일하게 배치



- Fps : 163 ~ 172

- 인스턴싱을 적용하여 배치했을 때의 프레임 수치는 약 2배 정도 향상된 것을 확인했습니다.

◆ 실제 게임 속에서의 Fps 결과

- 틀에서 스테이지를 구성하고 완성된 스테이지 데이터 파일을 로드했을 때는 Fps 62 ~ 74 를 확인할 수 있었습니다.



- 이는 디버그 모드로 빌드, 실행했을 때 확인된 Fps 수치입니다.

◆ 추가 최적화에 대한 고려

- 현재 상황에서 Release 모드로 빌드 실행했을 때 충분한 퍼포먼스를 보여주었으나, 조금 더 최적화를 할 수 있는지에 대한 고려를 하게 됩니다.
- 스테이지의 배경 오브젝트인 울타리나 바닥재 같은 정적 모델 종류가 늘어날 수록, 프레임 저하는 조금씩 발생할 가능성이 있다고 판단했습니다.

7-1

추가 최적화의 필요성

◆ 추가적인 최적화에 대한 고려

- 인스턴싱 렌더링으로 오브젝트 렌더링시, CPU 부담을 줄이고 GPU 연산을 통해 프레임을 향상 시켰으나, 그럼에도 스테이지 스케일이 확장될 수록 추가적인 최적화가 필요했습니다.
- 추가 최적화를 고민하며 내린 결론은 다음과 같습니다.

- 플레이어 시야에서 보이지 않는 오브젝트들은 렌더링 될 필요가 없었습니다.

절두체 컬링

- 마찬가지로 보이지 않는 몬스터들도 렌더링 및 애니메이션 동작할 필요가 없습니다.

트리거 형식의 렌더링
및 업데이트 수행

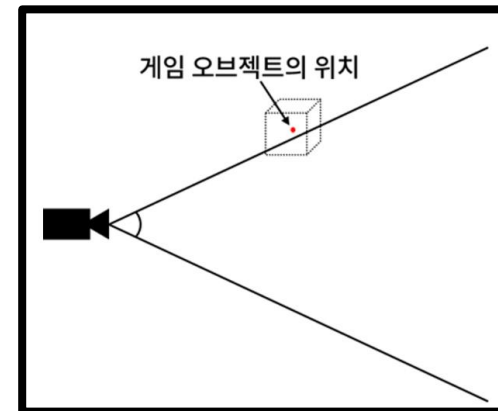
- 스테이지가 넓고 해당 스테이지를 구성하는 오브젝트가 많을 경우에는 하나의 맵을 나누어서 렌더링합니다.

전체 스테이지를 구역을
나누어 불러온 후, 렌더링

◆ 인스턴싱과 컬링의 관계



- 인스턴싱으로 렌더링되는 울타리 오브젝트 같은 경우, 수 백개가 각자의 위치에 맞게 렌더링되어 있지만, 플레이어가 보고있는 화면 안에서만 렌더링 되게한다.





7-2

AABB 바운딩 & 절두체 컬링 (1)

◆ 절두체 컬링 과정 설계

- 배치된 인스턴싱 오브젝트들의 AABB 바운딩 볼륨 설정
- 카메라 절두체 내부에 바운딩 볼륨들의 존재 유무 확인
- 존재한다면 그대로 렌더링, 없다면 컬링 진행

◆ AABB 바운딩 볼륨 좌표 범위 계산

- 초기화 시에, 각 인스턴스 메쉬들의 월드 좌표 중심과 스케일 값을 이용해, X, Y, Z축을 기준으로 최소/최대 좌표를 계산 후 컨테이너에 보관합니다.

```
XMFLOAT3 fMin = {
    fInstancePos.x - (m_vecInstanceScale[i].x * 0.5f),
    fInstancePos.y - (m_vecInstanceScale[i].y * 0.5f),
    fInstancePos.z - (m_vecInstanceScale[i].z * 0.5f)
};
XMFLOAT3 fMax = {
    fInstancePos.x + (m_vecInstanceScale[i].x * 0.5f),
    fInstancePos.y + (m_vecInstanceScale[i].y * 0.5f),
    fInstancePos.z + (m_vecInstanceScale[i].z * 0.5f)
};

m_vecAABBMin.push_back(fMin);
m_vecAABBMax.push_back(fMax);
```

◆ 카메라 절두체의 6개 평면 갱신

- 절두체는 카메라의 시야 공간이며, 6개의 평면으로 구성됩니다.
- 카메라의 뷰/투영 역행렬을 이용해 시야 공간을 월드 좌표계로 변환합니다.
- NDC 좌표 기준으로 8개의 꼭짓점을 월드 공간으로 변환합니다.

```
for (_uint i = 0; i < 8; i++)
{
    XMStoreFloat4(&m_vWorld_Points[i],
        XMVector3TransformCoord(XMLoadFloat4(
            &m_vOriginal_Points[i]), ProjMatrixInverse));

    XMStoreFloat4(&m_vWorld_Points[i],
        XMVector3TransformCoord(XMLoadFloat4(
            &m_vWorld_Points[i]), ViewMatrixInverse));
}
```

- 변환된 8개의 꼭짓점을 기반으로 6개의 평면을 생성합니다.

```
pPlanes[0] = XMPlaneFromPoints(p1, p5, p6); // Right
pPlanes[1] = XMPlaneFromPoints(p4, p0, p3); // Left
pPlanes[2] = XMPlaneFromPoints(p4, p5, p1); // Top
pPlanes[3] = XMPlaneFromPoints(p3, p2, p6); // Bottom
pPlanes[4] = XMPlaneFromPoints(p5, p4, p7); // Far
pPlanes[5] = XMPlaneFromPoints(p0, p1, p2); // Near
```




7-2

AABB 바운딩 & 절두체 컬링 (2)

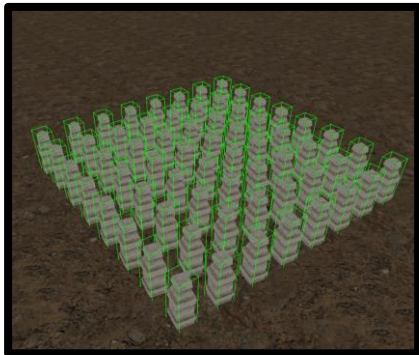
◆ AABB - 절두체 교차 판정

- 각 인스턴스의 8개 꼭짓점을 생성하고, 각각 절두체 평면과의 위치 관계를 검사합니다.

```
XMFLOAT3 fPoints[8] =
{
    XMFLOAT3(_fMin.x, _fMax.y, _fMin.z),
    XMFLOAT3(_fMax.x, _fMax.y, _fMin.z),
    XMFLOAT3(_fMax.x, _fMin.y, _fMin.z),
    XMFLOAT3(_fMin.x, _fMin.y, _fMin.z),

    XMFLOAT3(_fMin.x, _fMax.y, _fMax.z),
    XMFLOAT3(_fMax.x, _fMax.y, _fMax.z),
    XMFLOAT3(_fMax.x, _fMin.y, _fMax.z),
    XMFLOAT3(_fMin.x, _fMin.y, _fMax.z)
};
```

- _fMin 과 _fMax는 앞서 인스턴싱 오브젝트들의 초기화 단계에서 각 모델들의 AABB 바운딩 볼륨 좌표 범위 입니다.



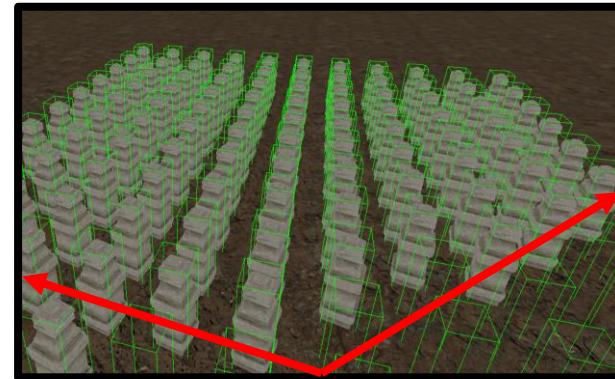
- 오브젝트의 AABB 바운딩 볼륨이 절두체 평면과 검사를 통해, 꼭짓점 8개가 내부에 있으면 그대로 렌더링, 모두 바깥에 존재한다면 렌더링을 하지 않습니다.

```
for (_uint i = 0; i < 6; ++i) {
    XMVECTOR vPlane = XMLoadFloat4(&m_vWorld_Planes[i]);
    _uint iCulledPointCount = {};

    for (_uint j = 0; j < 8; ++j) {
        XMVECTOR vPoint = XMLoadFloat3(&fPoints[j]);

        if (XMVectorGetX(XMPlaneDotCoord(vPlane, vPoint)) > 0.0f)
        {
            iCulledPointCount++;
        }
    }

    if (iCulledPointCount == 8)
        return false;
}
```

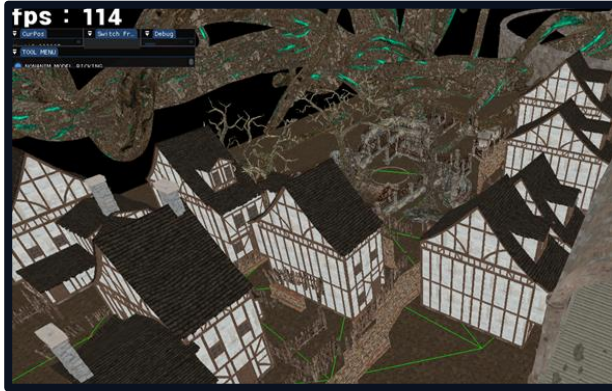




7-3

최적화 결과 (컬링 적용)

◆ 컬링 최적화 결과는 다음과 같습니다. (맵 툴)



- 컬링 전, Fps 110 ~ 120



- 울타리, 잔나무 등의 오브젝트 컬링 이후, Fps 15 ~ 165

◆ 인 게임 적용



- 인 게임 속 컬링 전 Fps 62 ~ 74



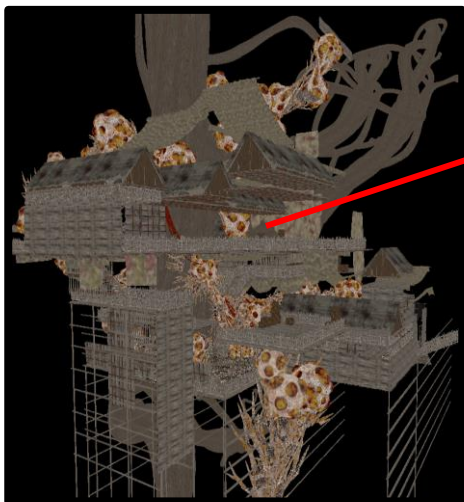
- 인 게임 속 컬링 이후 Fps 76 ~ 82



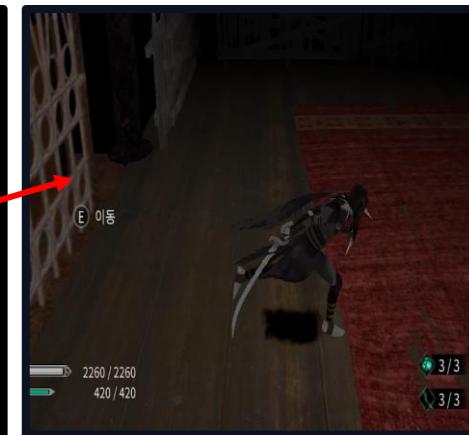
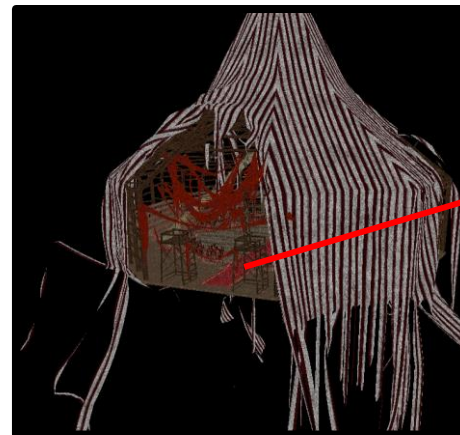
7-4

추가 최적화 아이디어

◆ 컬링 이외에도, 너무 큰 월드 맵들은 구역을 나누어서 트리거 형식으로 로드되게끔 설계했습니다.



- 스테이지 2의 맵을 총 2개의 섹션으로 구분했습니다.
- 보스 스테이지 맵은 플레이어가 마지막으로 도착하는 스테이지이기 때문에, 처음 부터 렌더링을 하지 않습니다.
- 보스 스테이지로 가는 엘리베이터 오브젝트와 상호작용 후, Fade In Out 효과로 넘어가며, 렌더링을 시작하게 됩니다.



- 사진 처럼 플레이어가 보스 스테이지에 도착하면 이전 스테이지들은 다시 돌아갈 일이 없기 때문에 삭제 처리를 하여 보스 스테이지에 퍼포먼스를 집중시켰습니다.

```
void CElevator_Door::OnCollision() {
    Activate_Fade(TRIGGER_TYPE::TT_FADE_IN, 1.f);

    for (auto& pIndex : m_vecLoadIndex) {
        Load_Objects(pIndex)
    }
    Delete_GameObject_From_Layer
        (m_iCurrentLevel, m_wstrCullingLayerTag);
}
```



END

결과 및 느낀 점

◆ 작업을 진행하면서, 구현도 좋지만 최적화 과정도 무엇보다 중요하다는 걸 체감했습니다.

- 열심히 구현을 했지만, 막상 그걸 보여주기 전에 심한 버벅임이 있다면, 아무 의미 없기 때문입니다.
- 최적화를 열심히 진행하며, 여러 블로그와 문서를 참고하며 학습했고, 결국 문제없이 팀 프로젝트를 마무리 할 수 있었습니다.
- 문제에 직면할 때는 다른 팀원들과의 토론도 나눠보고 의견을 주고 받는 일이 가장 좋은 것 같습니다.

◆ 참고했던 문서, 블로그 링크

- AABB - 프러스텀 컬링
<https://diesuki4.tistory.com/958>
- 인스턴싱
<https://devriripong.tistory.com/171>

앞으로도 협업 속에서 꾸준히 제 실력을 키워 나가겠습니다.

감사합니다.